Some teams see epics as much more, well, epic, and would place them above features. In that case, they're probably using a different, more granular, definition of the word "feature."

In any case, as you saw earlier, it's perfectly reasonable to have several levels of decomposition before you get to the user stories that you can implement in a sprint.

4.1.6 Not everything fits into a hierarchy

In real-world projects, not all requirements fit into the sort of neat hierarchical structures we've been talking about. Although this will work for many user stories, you'll sometimes come across a story that supports several features. For example, the "providing a secure password when registering" story we discussed in section 4.1.3 might relate to two features:

- Join the Frequent Flyer program online.
- Keep client data safe.

In that situation, you might say that the "join the Frequent Flyer program online" feature is a logical parent for this user story, but it's clearly also related to the cross-functional feature, "keep client data safe." This often happens when multiple stakeholders are involved. In this case, the business stakeholder wants travellers to be able to join the Frequent Flyer program, and the security stakeholder wants the feature to be delivered safely. There may be other stakeholders as well, such as compliance, legal, operations, and so forth.

Tags are a good way to handle this sort of situation. Many requirements-management and reporting tools let you use tags to organize your requirements, in addition to enabling a more conventional parent-child relationship. This way, you can present a relatively structured view of the main requirements hierarchy, but also keep track of any looser relationships. We'll look at using tags as part of the living documentation in part 3 of this book.

Once you have a better idea of which features you want to build, you need to flesh out your understanding of them. One of the best ways to do this is to talk through some concrete examples.

4.2 Illustrating features with examples

Examples are at the heart of BDD. In conversations with users and stakeholders, BDD practitioners use concrete examples to develop their understanding of features and user stories of all sizes, but also to flush out and clarify areas of uncertainty (see figure 4.13). These examples, expressed in language that business can understand, illustrate how the software should behave in very precise and unambiguous terms.

According to David Kolb's Experimental Learning theories, effective learning is a four-stage process.[1] In Kolb's model, we all start learning from concrete experiences

[1] David A. Kolb, *Experiential Learning: Experience as a Source of Learning and Development* (Prentice Hall, 1984).